PEOPLE'S DEMOCRATIC REPUBLIC OF ALGERIA

University of Sciences and Technology of Oran Mohamed-Boudiaf (USTO-MB)

Department of Computer Science

Smart Room : An Embedded Monitoring and Control System

Project Report

Presented by:
BENSALEM Loudjaine
BOUICHE Dounia
ZAOUIA Salsabil

Course:
Embedded Systems Technology (TSE)

2025 – 2026

Table of Contents

List of Figures

I. Introduction

Smart environments have become an important field in modern embedded systems, as they allow the monitoring and control of physical spaces through connected hardware and software components. In this context, smart room systems aim to improve indoor conditions by collecting data from sensors, processing this data, and displaying the system status in a clear and accessible way for the user.

The objective of this project is to design and implement a smart room prototype capable of monitoring and controling essential room parameters and presenting them in real time through a mobile application. The system is based on Arduino and integrates sensors, actuators, data storage, and mobile visualization. It allows the room to detect movement, count people, measure temperature and humidity, control basic actions, and display the results locally and remotely.

II. Conceptual Overview

The proposed system represents a smart room approach designed to monitor the state of a room and provide useful real-time feedback to the user. The project aims to create a simple but structured environment in which physical conditions such as temperature, humidity, light level, and occupancy can be captured by sensors, processed by an embedded controller, stored in a database, and finally visualized through a mobile application.

At a conceptual level, the room includes several sensing and actuating components. The sensors are used to detect movement, measure temperature and humidity, detect light level, and support the people counting process. The system also includes actuators such as LEDs, a heater indicator, an LCD display, and a servo motor used to represent an automatic door mechanism.

The user interacts with the system through a mobile application that displays the current status of the room. This includes the number of people inside the room, the available seats, the temperature, the humidity, the heating state, and the current date and time. The conceptual goal is therefore not only to collect raw sensor data, but also to transform this data into meaningful information that can be easily understood by the user.

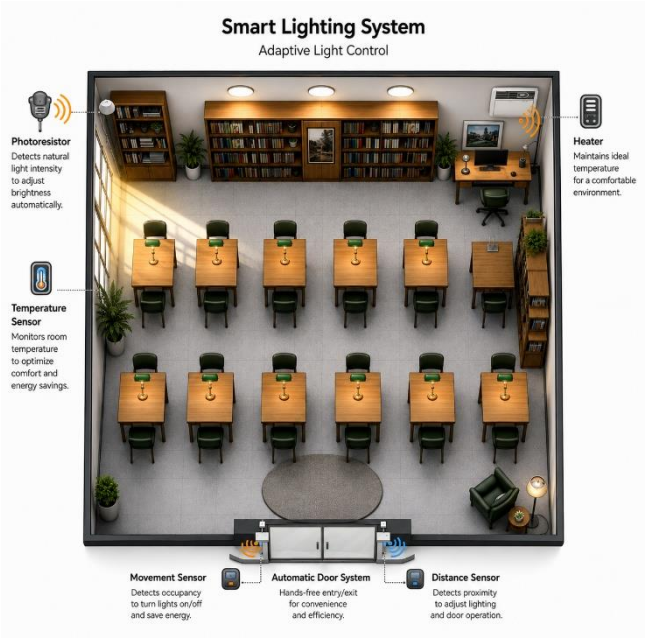

Figure 1: Conceptual Overview of the Smart Room System

III. System Architecture

The system uses a multi-layer architecture in order to separate the responsibilities of each component and make the solution easier to understand, maintain, and extend. Each layer has a specific role in the complete data flow, starting from the physical environment and ending with the mobile interface.

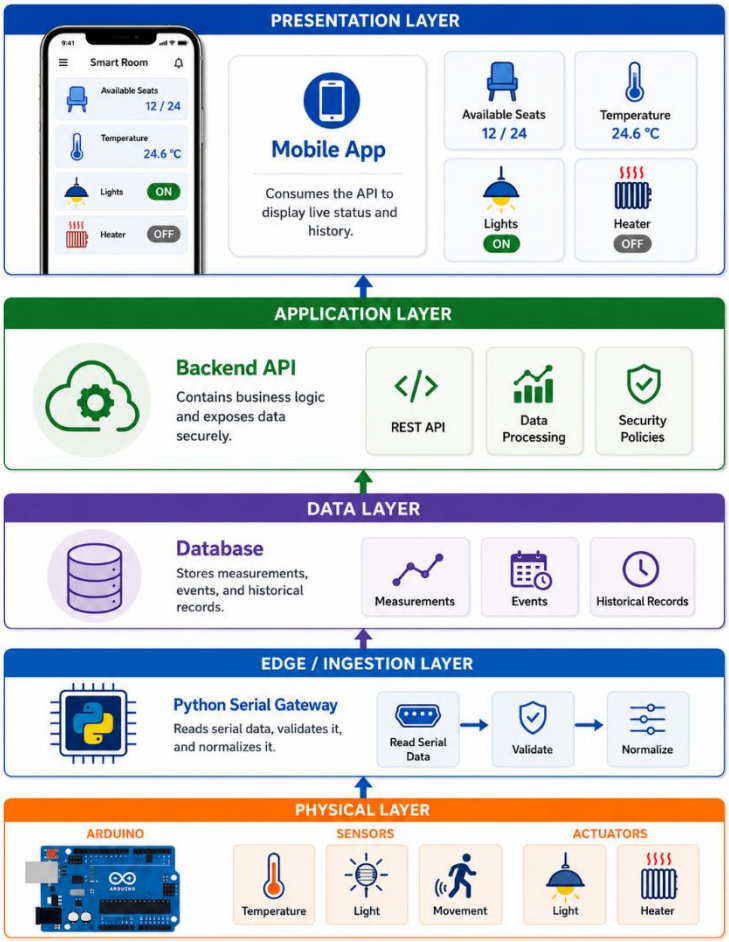

Figure 2: Layered Architecture of the Smart Room System

III.1 Physical Layer

The physical layer contains the hardware components used in the room prototype. It includes:

- Arduino Uno board: acts as the central controller for the connected sensors and actuators.
- The sensors: - PIR motion sensors
  - A DHT11 temperature and humidity sensor
  - A photoresistor for light detection,

These sensors collect information about the room environment and generate signals that represent the physical state of the room.

- The actuators:- LEDs
  - LCD I2C display
  - servo motor.

III.2 Edge / Ingestion Layer

The edge layer is responsible for receiving and interpreting the raw data produced by the Arduino. In this project, a Python script is used as the ingestion component. Its role is to read the serial data transmitted by the Arduino through the USB connection.

It acts as a bridge between the embedded system and the mobile application. It receives structured data from Arduino, organizes the information, and prepares it for storage.

III.3 Data Layer

The data layer stores the measurements and events generated by the system in SQLite database.

The database contains information such as timestamp, people count, temperature, humidity, heater state, and door state.

III.4 Application Layer

The application layer contains the backend logic of the application.

III.5 Presentation Layer

A mobile application. It is the visual interface used by the end user to monitor the room.

The app displays the main information collected by the lower layers, including people count, available seats, temperature, humidity, heating state, and real-time date and time.

This layer transforms technical data into a simple interface that can be understood without specialized knowledge.

IV. Implementation and Integration

The implementation of the system combined embedded hardware, Arduino programming, a Python backend, a database, a Flask API, and a mobile application. Each part was developed to support the layered architecture described previously.

IV.1 Embedded System Implementation

The hardware setup is based on an Arduino Uno board connected to the main sensors and actuators. The Arduino continuously reads sensor values and applies the system logic.

The main hardware components used are:

- Arduino Uno as the main controller.
- Breadboard and jumper wires for circuit connections.
- USB cable for power supply and serial communication.
- Resistors to limit current and protect components.
- PIR motion sensors to detect entry and exit movement.
- DHT11 sensor to measure temperature and humidity.
- Photoresistor to detect light level.
- Servo motor to control the automatic door.

- LCD I2C display to show room information locally.
- LEDs as visual indicators for lighting and heating.

The Arduino code was developed using the Arduino IDE. The main libraries used are DHT.h for the DHT11 sensor, Wire.h for I2C communication, LiquidCrystal_I2C.h for the LCD display, and Servo.h for controlling the servo motor.

The implemented functions include:

- Sensor reading
- People counting
- Capacity limit control
- Automatic door control
- Light control
- Heater control
- LCD display update
- Serial data transmission.

The people counting logic is based on two motion sensors. If the external sensor detects movement before the internal sensor, the system considers that a person is entering. If the internal sensor detects movement before the external sensor, the system considers that a person is leaving. The system also includes a maximum capacity condition. When the room reaches the maximum number of people, entry is blocked while exit remains allowed.

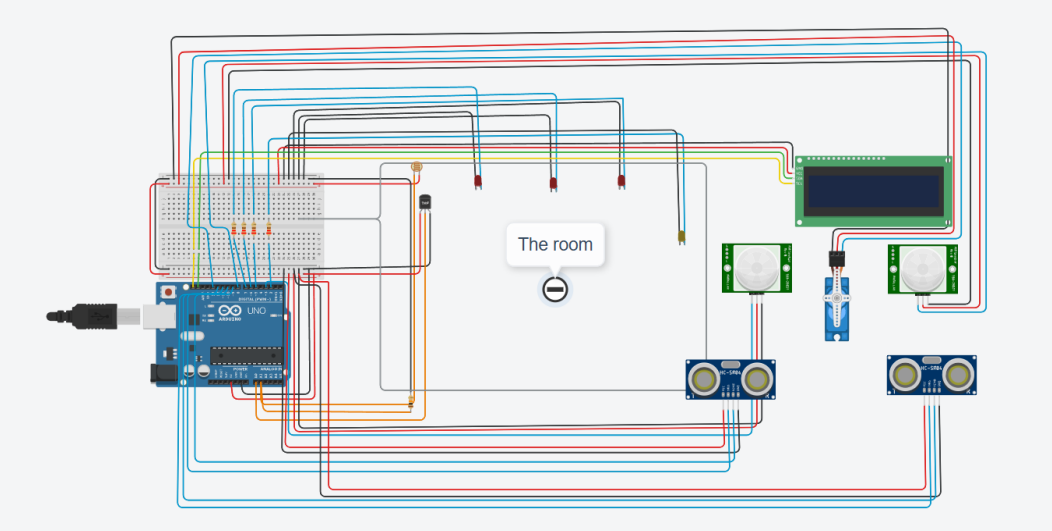

Figure 3: Embedded System Circuit Simulation in Tinkercad

IV.2 Software Integration and Data Pipeline

The software integration connects the embedded system to the mobile application.

- The Arduino sends structured data through serial communication. The data includes the people count, temperature, humidity, heater state, door state, and room availability status.
- The Python backend receives this data from the serial port, extracts the values, and stores them in the SQLite database.
- The Flask API is then used to make the latest values available to the mobile application. The app requests the latest data from the /latest endpoint and receives the response in JSON format.

The general data pipeline is:

Arduino → Serial Communication → Python Backend → SQLite Database → Flask API → Android Application

IV.3 Mobile Application Implementation

The mobile application was developed using Kotlin in Android Studio. It represents the presentation layer of the system and allows the user to monitor the smart room in real time.

It displays the number of people inside the room, available seats out of 20 (as an example), temperature, humidity, heating state, and current date and time.

The interface was designed to be simple and readable. Instead of showing raw data only, it presents the information as clear room indicators that can be understood directly by the user.

V. Results and Discussion

The final prototype demonstrates the complete data flow from sensing to visualization. When the system is running, the Arduino collects data from the sensors, processes it, controls the actuators, displays local information on the LCD, and sends the data through serial communication.

The backend receives the Arduino data, stores it in the SQLite database, and makes it available to the mobile application through the Flask API. The mobile app then displays the room status in real time.

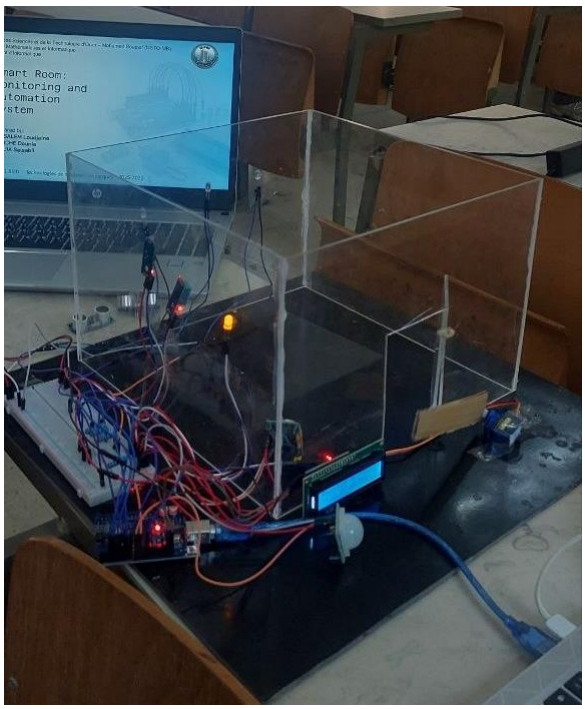

Figure 4: Working Prototype of the Smart Room System

Figure 5: LCD Display Showing Local Room Information

The system was able to show the number of people inside the room, calculate the available seats, measure temperature and humidity, display the heating state, and update the mobile application in real-time. The LCD also displayed the main values locally.

The results confirm that the layered architecture is effective because each part of the system has a clear role. The Arduino handles the embedded control, Python manages data reception and storage, Flask exposes the data, and the Android application presents the information to the user.

From a functional point of view, the prototype behaves as expected. It receives input from sensors, applies the control logic, stores the values, and displays the results through both local and mobile interfaces.

However, the prototype also has some limitations. Since it is for an academic purpose, some real devices are represented by LEDs. For example, the heater is represented by an LED instead of a real heating device. Also, the PIR sensors can be sensitive, so their positioning and adjustment are important for better detection.

```
Received: DATA,4,23.80,71.00,ON,CLOSED,FULL
Saved | 2026-05-21 12:52:19 | People: 4 | T: 23.8 C | H: 71.0% | Heater: ON | Door: CLOSED | Status: FULL
Received: External: 0 | Internal: 0 | People: 4 FULL | Temperature: 23.80 C | Humidity: 71.00 % | Heater:
 ON | Door: CLOSED
Received: DATA,4,23.80,71.00,ON,CLOSED,FULL
Saved | 2026-05-21 12:52:22 | People: 4 | T: 23.8 C | H: 71.0% | Heater: ON | Door: CLOSED | Status: FULL
Received: External: 1 | Internal: 0 | People: 4 FULL | Temperature: 23.80 C | Humidity: 71.00 % | Heater:
 ON | Door: CLOSED
Received: DATA,4,23.80,71.00,ON,CLOSED,FULL
Saved | 2026-05-21 12:52:24 | People: 4 | T: 23.8 C | H: 71.0% | Heater: ON | Door: CLOSED | Status: FULL
Received: ROOM FULL. Entry blocked.
Received: External: 0 | Internal: 1 | People: 4 FULL | Temperature: 23.80 C | Humidity: 71.00 % | Heater:
 ON | Door: CLOSED
```

Figure 6: Python Terminal Receiving and Saving Data

```
(1920, '2026-05-21 00:02:35', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1919, '2026-05-21 00:02:33', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1918, '2026-05-21 00:02:30', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1917, '2026-05-21 00:02:28', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1916, '2026-05-21 00:02:25', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1915, '2026-05-21 00:02:23', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1914, '2026-05-21 00:02:20', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1913, '2026-05-21 00:02:18', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1912, '2026-05-21 00:02:15', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1911, '2026-05-21 00:02:13', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1910, '2026-05-21 00:02:10', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1909, '2026-05-21 00:02:08', 1, 25.3, 65.0, 'OFF', 'CLOSED')
(1908, '2026-05-21 00:02:05', 1, 25.3, 65.0, 'OFF', 'CLOSED')
```

Figure 7: Database

Figure 8: Mobile Application Interface

VI. Challenges and Future Improvements

VI.1 Challenges Encountered

Several challenges were encountered during the development of the project such us:

- COM port conflict between Arduino IDE and Python. Since only one program can use the serial port at a time, the Arduino Serial Monitor had to be closed when running the Python backend.
- The people counting logic also required adjustments. At the beginning, the system could count the same movement more than once. This was improved by using sequence logic, cooldown timing, and conditions to avoid double counting.
- The PIR motion sensors are too sensitive and could detect movement from a wider area than expected. This required physical adjustment of the sensor direction and sensitivity.
- The Android application connection with the local Flask server was another challenge. ADB reverse was used during testing to allow the phone to access the server running on the computer.

VI.2 Future Improvements

Several improvements can be added in future versions of the project:

- the Arduino Uno can be replaced by a Wi-Fi-enabled board such as ESP32 or ESP8266. This would allow direct wireless communication and without dependency on USB serial communication.
- An administrator mode can be added to the mobile application. This would allow the administrator to manually control some room actions, such as opening the door, activating the heater, or managing room access or lighting.
- Safety features can also be integrated by adding flame, smoke, or gas sensors. In case of danger, the system could send alerts to the mobile application.
- Future versions can add CO_2 or air pollution sensors and control ventilation or air purification systems.
- Real actuator control can be added using relay modules. Instead of representing the heater or lights with LEDs, the system could control real electrical devices such as lamps, heaters, fans, or ventilation systems.

VII. Conclusion

This project presented the design and implementation of a smart room monitoring and automation system. The prototype combines embedded hardware, software processing, data storage, and mobile visualization.

The final system demonstrates how a traditional room can be transformed into a connected smart environment. It provides real-time information about room occupancy and environmental conditions, while also supporting basic automation such as door control and heating indication.

Although the system remains a prototype, it provides a strong base for future improvements such as Wi-Fi communication, real actuator control, safety alerts, air quality management, and cloud-based IoT monitoring.

Appendix A: Tinkercad Simulation Link

https://www.tinkercad.com/things/hb6XKCDYtRV-smart-room?sharecode=S4YNLvbtr4a2TvJbT24urnJHxnpiuZ8lL5D2Yparf5E